

COMP.SE.140-2025-2026-1

Continuous Development and Deployment

Assignment Report

By

Ahmad Ali Khan Shirwany

Student ID: 151485383

Basic information about the platform I have used:

Platform Used: Virtual Machine

Operating System details:

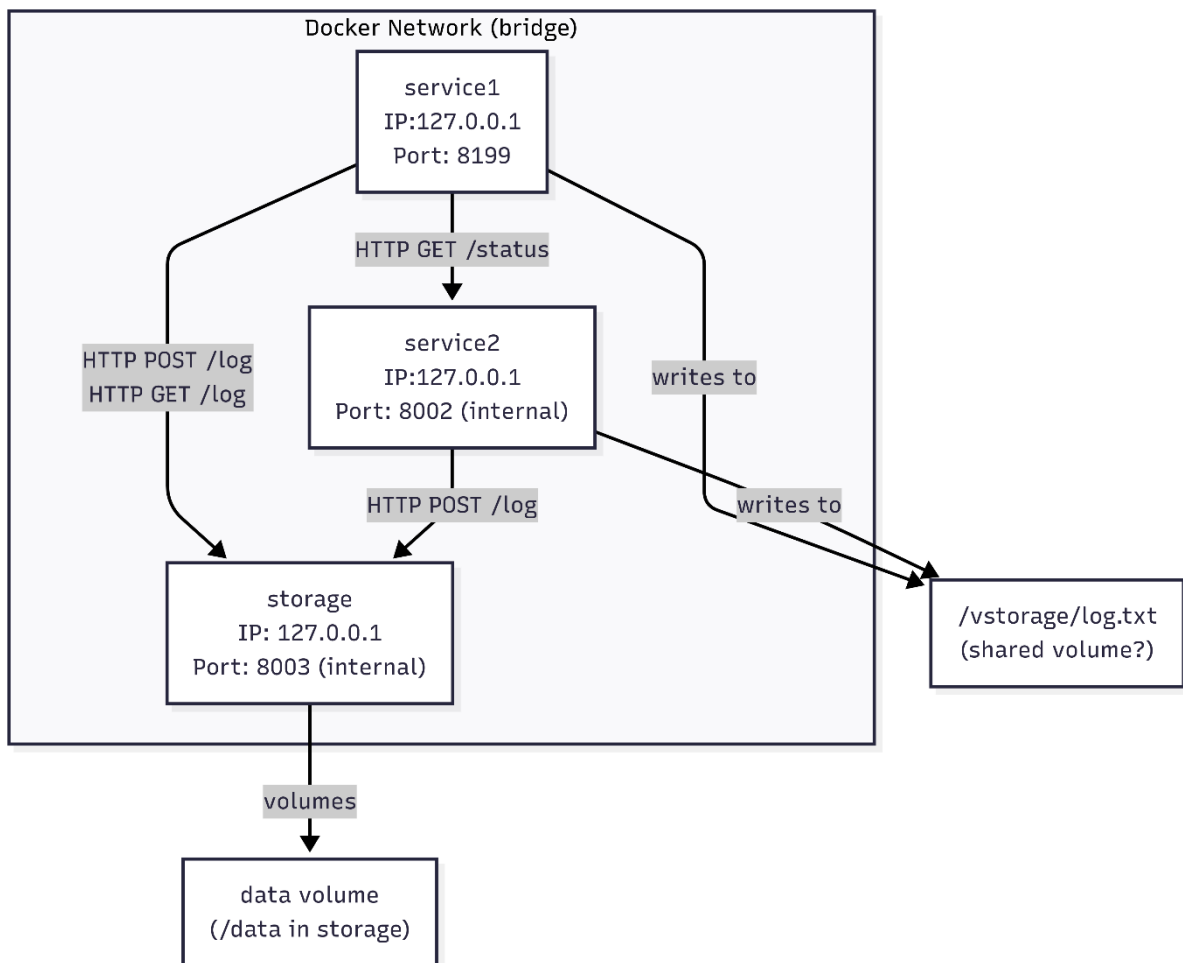
PRETTY_NAME="Ubuntu 24.04.3 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.3 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo

Version numbers of Docker and Docker-Compose

Client: Docker Engine - Community
Version: 28.4.0
API version: 1.51
Go version: go1.24.7
Git commit: d8eb465
Built: Wed Sep 3 20:57:32 2025
OS/Arch: linux/amd64
Context: default

Server: Docker Engine - Community
Engine:
Version: 28.4.0
API version: 1.51 (minimum version 1.24)
Go version: go1.24.7
Git commit: 249d679
Built: Wed Sep 3 20:57:32 2025
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: 1.7.27
GitCommit: 05044ec0a9a75232cad458027ca83437aae3f4da
runc:
Version: 1.2.5
GitCommit: v1.2.5-0-g59923ef
docker-init:
Version: 0.19.0
GitCommit: de40ad0

A diagram showing the services, network and storage



Analysis of the content of the status records. Which disc space and uptime you actually measured? How relevant are those measurements? What should be done better?

The services measure the system's overall uptime (time since the host machine booted) and free disk space on the root filesystem using `df`, but not container-specific values. This makes the uptime only moderately useful, since container uptime would give better insight into service restarts, and the disk space is mislabeled as "Free Memory" when it is actually free disk space. These measurements are somewhat relevant for host-level monitoring but limited for service-level health. To improve uptime should be tracked per container, disk usage should be measured on relevant volumes, and additional metrics such as memory, CPU, and network usage should be included using proper monitoring tools.

Analysis and comparison of the persistent storage solutions – what is good and what is bad in each solution?

In this setup, Docker named volumes are used for storing data. The “vstorage” volume is meant for shared logs between the two services, and the data volume is used by the storage service to keep logs from POST requests. The good side is that data is saved even if containers are restarted or rebuilt, volumes are isolated from the host (so safer), and they are easy to back up or move with Docker. They also keep things simple since Docker manages everything. The bad side is that accessing the data directly from the host is not easy without Docker commands, cleaning up volumes can be tricky, and if not careful data can be lost.

Storage	Details
Volumes used	• vstorage: shared logs between service1 and service2 • -data: logs from POST requests in storage service
Good points	• Data survives restarts/rebuilds • Isolated from host (safer) • Easy to back up or move with Docker • Simple to manage (Docker handles storage)
Bad points	• Hard to access directly from host (need Docker commands) • Cleanup can be tricky, risk of data loss
Comparison	• Named volumes = safer, more portable • Host binds = easier for quick debugging (files visible directly on host)

Teacher’s instruction for cleaning up the persistent storage

```
git clone -b exercise1 https://github.com/ahmadshirwany/DevOpsExercise_1.git
```

```
cd DevOpsExercise_1
```

```
docker-compose up --build
```

```
curl http://localhost:8199/status
```

instructions for cleaning up

```
docker-compose down -v
```

```
docker network rm devopsexercise_1_default
```

```
cd ..
```

```
rm -rf DevOpsExercise_1
```

What was difficult?

The difficult parts were mainly related to handling volumes. It was hard to identify which volumes existed and what they contained since we couldn't check them directly from the host. There were also path mismatches that caused problems, like logs not being saved in the right or because vstorage was not mounted properly. Inspecting the data inside a volume was also tricky, as we had to run temporary containers just to read the files. Finally, cleaning up volumes safely was challenging because we had to make sure no running containers were using them to avoid losing data.

What were the main problems?

The main problems were mostly about how volumes and paths were handled. The vstorage volume was defined but not actually mounted to service1 or service2, even though the code expected it, which could cause "no such file or directory" errors or prevent data from being saved. There were also path mismatches, like storage.py using /storage while the volume was mounted to /data, so logs might not save or load correctly. Volume management was unclear since there were no scripts or instructions for backup and cleanup, making manual commands easy to mess up. Additionally, logs were split between vstorage and storage, causing duplication and inconsistencies, and the services didn't check if volumes were writable, which could lead to silent failures.